Trabalho Prático 02

João Gabriel Guimarães Alves Vaz

Departamento de Ciência da Computação da UFMG (DCC/UFMG)
Belo Horizonte, Minas Gerais - Brasil
Matrícula: 2023099166
jgvaz@ufmg.br

2025

1 Introdução

O problema computacional a ser resolvido consiste em determinar o menor número de soldados necessário para proteger uma capital em um mapa retangular. Esse mapa é representado por uma matriz de inteiros, onde cada célula contém um valor positivo que indica a quantidade de soldados exigidos para defendê-la ou o valor zero, que representa uma montanha intransponível. Invasores podem entrar no reino por qualquer uma das posições de borda e se deslocar apenas em quatro direções: cima, baixo, esquerda e direita. O desafio é bloquear, com o menor custo total, todos os caminhos possíveis que levam da borda até a capital, impedindo o avanço inimigo. Para isso, o problema é modelado como um grafo direcionado com capacidades e resolvido através de um algoritmo de fluxo máximo, cuja resposta corresponde ao custo do corte mínimo — isto é, a menor soma de soldados que garante o isolamento completo da capital.

Abaixo está um resumo do que será abordado em cada uma das sessões desse documento.

- Na sessão 1, há uma introdução ao tema e este sumário;
- Na sessão 2, há detalhes da modelagem;
- Na sessão 3, há a solução para o poblema;
- Na sessão 4, há uma análise de complexidade;
- Na sessão 5, há considerações finais e a conclusão;
- Na sessão 6, há as referências.

2 Modelagem

Para modelar o problema, representamos o mapa como um grafo direcionado com capacidades, onde cada posição acessível da matriz (isto é, qualquer célula com valor maior que zero) é transformada em dois vértices distintos: um vértice de entrada e um vértice de saída. Esses dois vértices são conectados por uma aresta de capacidade igual ao número de soldados necessários para defender aquela posição, conforme especificado na matriz. Essa duplicação de vértices permite controlar com precisão o custo de bloquear uma posição específica, sem impedir a livre circulação entre posições vizinhas.

Além disso, para representar o movimento dos invasores, conectamos o vértice de saída de cada célula aos vértices de entrada de suas vizinhas acessíveis (acima, abaixo, esquerda e direita), por meio de arestas com capacidade infinita. Isso garante que o inimigo possa se mover livremente entre as posições, exceto quando a passagem for explicitamente bloqueada por soldados.

O grafo também inclui dois vértices especiais: uma pseudo-fonte, que se conecta a todas as posições da borda do mapa (representando os pontos de entrada do inimigo), e uma pseudo-destino, que está ligada à posição da capital (representando o ponto a ser protegido). A partir dessa modelagem, o problema é resolvido aplicando o algoritmo de Ford-Fulkerson para fluxo máximo, utilizando busca em largura (BFS) para encontrar caminhos aumentantes. Com isso, conseguimos calcular o corte mínimo entre a fonte e o destino, o que nos fornece o número mínimo total de soldados necessários para proteger a capital.

3 Solução

A solução do problema foi estruturada em duas fases principais: a construção do grafo que representa o mapa e a aplicação do algoritmo de fluxo máximo de Ford-Fulkerson sobre esse grafo.

Na construção do grafo, cada célula acessível da matriz é representada por dois vértices: um de entrada e um de saída. A primeira etapa consiste em conectar esses dois vértices por uma aresta com capacidade igual ao número de soldados necessários para defender a posição, conforme representado na matriz. Esse procedimento é descrito pelo seguinte pseudocódigo:

Algorithm 1 Criação de vértices gêmeos com capacidade de defesa

```
for all posição (i, j) da matriz do
    if matriz[i][j] > 0 then
        u_in ← criar_vértice(i, j, "in")
        u_out ← criar_vértice(i, j, "out")
        adicionar_aresta(u_in, u_out, matriz[i][j])
    end if
end for
```

A segunda etapa conecta cada vértice de saída às entradas dos seus vizinhos válidos (cima, baixo, esquerda, direita) por arestas de capacidade infinita, permitindo o deslocamento irrestrito do inimigo entre posições acessíveis:

Algorithm 2 Conexão com vizinhos com capacidade infinita

```
for all posição (i, j) da matriz do
    if matriz[i][j] > 0 then
        u_out ← obter_vértice(i, j, "out")
        for all (dx, dy) ∈ {(−1, 0), (1, 0), (0, −1), (0, 1)} do
            (ni, nj) ← (i + dx, j + dy)
            if matriz[ni][nj] > 0 then
                v_in ← obter_vértice(ni, nj, "in")
                adicionar_aresta(u_out, v_in, ∞)
            end if
        end for
    end if
end for
```

Além disso, dois vértices especiais são criados: a pseudo-fonte (SOURCE), que representa todas as bordas do mapa (possíveis pontos de entrada dos inimigos), e a pseudo-destino (SINK), que representa a capital. Esses vértices são únicos e não são divididos em in/out como as demais posições.

Algorithm 3 Criação da pseudo-fonte e pseudo-destino

```
SOURCE ← novo_vértice()
SINK ← novo_vértice()
```

A fonte é então conectada a todos os vértices de entrada nas bordas do mapa que não são montanhas:

Algorithm 4 Conexão da fonte com as posições fronteiriças

for all posição (i, j) da borda do mapa **do**
  if matriz$[i][j] > 0$ **then**
    $v_{in} \leftarrow$ obter_vértice$(i, j,$ "in")
    adicionar_aresta(SOURCE, v_{in}, ∞)
  end if
end for

A capital é conectada diretamente ao destino:

Algorithm 5 Conexão da capital ao destino

$v_{out} \leftarrow$ obter_vértice(capital[0], capital[1], "out")
adicionar_aresta(v_{out}, SINK, ∞)

Com o grafo construído, o algoritmo de Ford-Fulkerson é executado. A cada iteração, o algoritmo realiza uma busca em largura (BFS) para encontrar um caminho aumentante entre a SOURCE e a SINK. Em seguida, determina o gargalo (menor capacidade ao longo do caminho), atualiza as capacidades residuais das arestas e acumula o fluxo total:

Algorithm 6 Algoritmo de Ford-Fulkerson

$fluxo \leftarrow 0$
$parent[v] \leftarrow -1$, para todo $v \in V$
$visited \leftarrow$ bfs(SOURCE, $parent$)
while $visited$[SINK] é verdadeiro **do**
  $path_flow \leftarrow \infty$
  $v \leftarrow$ SINK
  while $v \neq$ SOURCE **do**
    $u \leftarrow parent[v]$
    $path_flow \leftarrow \min(path_flow, capacidade[u, v])$
    $v \leftarrow u$
  end while
  $v \leftarrow$ SINK
  while $v \neq$ SOURCE **do**
    $u \leftarrow parent[v]$
    $capacidade[u, v] \mathrel{-}= path_flow$
    $capacidade[v, u] \mathrel{+}= path_flow$
    $v \leftarrow u$
  end while
  $fluxo \mathrel{+}= path_flow$
  $visited \leftarrow$ bfs(SOURCE, $parent$)
end while
return $fluxo$

A execução se encerra quando não há mais caminhos aumentantes disponíveis. Segundo o teorema do fluxo máximo e corte mínimo, o valor retornado por essa função corresponde exatamente à soma das capacidades do menor conjunto de arestas que separa a fonte do destino — ou seja, ao número mínimo de soldados necessários para impedir completamente o acesso à capital.

4 Análise de Complexidade

A solução proposta utiliza uma modelagem por grafos e aplica o algoritmo de Ford-Fulkerson com busca em largura (BFS) para encontrar caminhos aumentantes. A complexidade assintótica da solução pode ser analisada separadamente em termos de tempo e memória.

Tempo

A cada iteração do algoritmo de Ford-Fulkerson, realizamos uma busca em largura para encontrar um caminho aumentante no grafo residual. Em cada chamada da BFS, percorremos todas as arestas do grafo, o que leva tempo proporcional a $\mathcal{O}(E)$, onde E é o número total de arestas.

O número total de iterações no pior caso é proporcional ao valor do fluxo máximo F, pois a cada iteração garantimos ao menos 1 unidade de fluxo sendo enviada. Logo, a complexidade total de tempo da solução é:

$$\mathcal{O}(E \cdot F)$$

Neste problema, cada célula do mapa com valor diferente de zero se transforma em dois vértices e pode se conectar a até quatro vizinhos, além das arestas que ligam à pseudo-fonte e pseudo-destino. Assim, o número de vértices V e de arestas E são ambos proporcionais ao tamanho do mapa. Se o mapa for de tamanho $n \times m$, então:

$$V = \mathcal{O}(n \cdot m), \quad E = \mathcal{O}(n \cdot m)$$

O valor de F, o fluxo máximo, pode no pior caso ser a soma de todos os valores positivos da matriz, que é limitada por $\mathcal{O}(n \cdot m \cdot k)$, onde k é o maior valor possível de soldados em uma célula. No entanto, como esse valor depende da entrada, F costuma ser muito menor.

Memória

A estrutura utilizada para armazenar o grafo inclui:

- Lista de adjacência para cada vértice: $\mathcal{O}(V + E)$;
- Dicionário de capacidades residuais entre pares de vértices: $\mathcal{O}(E)$;
- Vetores auxiliares como o vetor de pais (`parent`) e de visitados (`visited`) usados na BFS: $\mathcal{O}(V)$.

Portanto, a complexidade de espaço da solução é:

$$\mathcal{O}(V + E) = \mathcal{O}(n \cdot m)$$

5 Considerações finais

A realização deste trabalho prático foi uma experiência bem interessante e, ao mesmo tempo, desafiadora. A parte mais tranquila foi lidar com a entrada e leitura da matriz — isso envolveu apenas operações básicas de programação, que já estavam bem assimiladas ao longo da disciplina. O que realmente exigiu mais esforço foi a modelagem do problema como grafo. Foi necessário entender bem como transformar cada posição do mapa em dois vértices distintos (entrada e saída), como criar as arestas com capacidade infinita entre vizinhos e como organizar todos esses vértices usando identificadores únicos. Essa etapa exigiu bastante atenção, principalmente para evitar erros de conexão ou de lógica na hora de montar o grafo completo. Outra parte que deu trabalho foi implementar corretamente o algoritmo de Ford-Fulkerson. Mais do que apenas escrever o código, foi importante entender como o grafo residual funciona, como as capacidades vão sendo atualizadas ao longo do tempo, e como garantir que o fluxo realmente represente o número mínimo de soldados necessários. No fim, o trabalho ajudou bastante a consolidar os conceitos de fluxo máximo e corte mínimo, e mostrou na prática como eles podem ser aplicados em um problema contextualizado e com uma narrativa interessante. Foi desafiador, mas também recompensador, ver tudo funcionando como esperado.

6 Referências

- Almeida, J. (2025). Slides virtuais da disciplina de algoritmos 1. Disponibilizado via moodle. Departamento de Ciência da Computação. Universidade Federal de Minas Gerais. Belo Horizonte.